

Laravel Installation and Project Structure

Set up one working application before adding features.

Learning Goals

- Run a working Laravel application and open its welcome page.
- Locate routes, controllers, models, migrations, and `.env` in the project.
- Explain what Composer, Artisan, migrations, and Sanctum each do.

Set Up Your Environment

Read the `laravel_install.pdf` first, as it is the the single source for:

- PHP, Composer, and MySQL installation (macOS and Ubuntu/WSL2);
- version checks and the course database settings;
- creating the `student-api` project and starting the server;
- troubleshooting.

Windows: run all commands inside WSL2/Ubuntu.

Your Practice Project

The setup guide creates `student-api` (see *Step 3: Create the Laravel Project*) and shows how to run it (see *Start Laravel*).

- Once the server is running, open the welcome page in your browser.

Stop the server with **Ctrl+C**.

- Use a fresh directory for each lesson's generated example.

Files You Will Use

Location	Purpose
<code>routes/</code>	Match requests to code
<code>app/Http/Controllers/</code>	Handle requests
<code>app/Models/</code>	Work with stored data
<code>database/migrations/</code>	Define database changes
<code>.env</code>	Local application settings

php artisan, migration, and sanctum

Laravel supports tools for automation (artisan), DB (migration), and security (Sanctum).

Artisan

`artisan` is Laravel's command-line tool.

- Use commands in the form `php artisan <command>` for routine development tasks such as starting the server, generating code, managing configuration, and working with the database.

Migration

A `migration` records a database structure change in code, such as creating a table or adding a column.

- This lets every developer and deployment environment apply the same changes in the same order.
- `php artisan migrate` applies any migrations that have not yet run, keeping the database structure aligned with the project code.

Sanctum

Sanctum stores personal access tokens in this table.

- Clients send a token as `Authorization: Bearer <token>`, and Sanctum validates it to authenticate the user.
- Sanctum provides Bearer-token authentication; your app must still enforce authorization.

Artisan Commands

Artisan is Laravel's command-line tool for common project tasks.

- In Module 1, we handled server behavior directly in PHP.
- Artisan streamlines routine Laravel operations.

Next, use Artisan to enable API routes and apply their database migration.

Enable API Routes

In the project directory, if API setup has not been done:

```
php artisan install:api  
php artisan migrate  
php artisan route:list --path=api
```

This sets up API routing and Sanctum. You still write your endpoints.

- The course scripts may already perform these steps.

Useful Commands

```
php artisan about  
php artisan route:list  
php artisan config:clear
```

- `about` summarizes the application
- `route:list` shows registered routes
- `config:clear` clears cached configuration

Run `php artisan list` to see more commands.

Database Settings in `.env`

The database values are defined once, in the `laravel_install.pdf` (see *Course Database Settings*).

- Scripts that use the database, such as `run1-4.sh`, write them to `.env` for you.
- The setup scripts and Laravel use the same variable names:

Variable	Course value
<code>DB_DATABASE</code>	<code>studentdb</code>
<code>DB_USERNAME</code>	<code>ase230</code>
<code>DB_PASSWORD</code>	<code>ase230pass</code>

Do not commit `.env` or real passwords.

Before Moving On

- Open the welcome page.
- Locate routes, controllers, and models.
- Verify MySQL settings and run migrations.
- Explain what Composer and Artisan each do.